Hands-On Perception 2026

Lab 1 : Generation and Use of ArUco Markers with OpenCV

Josep Forest, Valerio Franchi, Nuno Gracias

February, 2026

Part 2 - Generation of ArUco Markers

In this part 2 of the lab, you are going to develop a simple program that generates both a single ArUco marker and a board of them.

In the following parts 3, 4 and 5, you will learn how to detect ArUco markers, calibrate cameras using ArUco boards and finally, how to use the markers for augmented reality and distance measurement.

2 Generation of ArUco markers

An ArUco marker consists of the graphical representation of a square that contains a bidimensional binary code determining its identifier. The square is, in fact, an n x n matrix with a black or white cell in each of its (row, column) positions. The distribution of the white cells on the black square does not follow a random pattern, but it is generated according to a particular dictionary chosen among the ones available in OpenCV. Each of the n x n cells in the ArUco marker represents one bit of the bidimensional binary code.

One constraint that is always forced in the generation of ArUco markers is that all cells in the square frame are always black.

Implement two C++ programs using OpenCV, called from the command line according to the following two sections:

2.1 Program implementation: generate_marker

The program generates a single ArUco marker. Four parameters are expected when executing the program:

1. dictionary (alphanumerical)
2. id of the marker in the dictionary (numerical)
3. size of the marker in pixels (numerical)
4. name of the PNG image file containing the ArUco marker (alphanumerical)

This is an example of how a call to the program from command line could be:

```
~build$ ./generate_marker DICT_ARUCO_ORIGINAL 25 200 marker.png
```

After the program is executed, the file **marker.png** should contain an image similar to the one shown in Figure 2.1.

NOTE: In order to specify the ArUco dictionary, you may use the OpenCV naming style, for example: DICT_6x6_250.
NOTE: It is strongly recommended to generate the ArUco marker surrounded by a white frame, since OpenCV uses the contrast between black and white in order to detect its borders.

Figure 2.1: ArUco marker generated using the original ArUco dictionary, id 25

2.2 Program implementation: generate_board

- The program generates a matrix of ArUco markers
- 6 parameters are expected when executing the program:
  1. number of rows of the ArUco matrix (numerical)
  2. number of columns of the ArUco matrix (numerical)
  3. dictionary (alphanumerical)
  4. marker size in pixels (numerical)
  5. separation between markers in pixels (numerical)
  6. name of the PNG image file containing the matrix (alphanumerical) This is an

example of how a call to the program from command line could be:

```
~build$ ./generate_board 2 4 DICT_ARUCO_ORIGINAL 200 80 board.png
```

After the program is executed, the file **board.png** should contain an image similar to the one shown in Figure 2.2.

NOTE: In order to specify the ArUco dictionary, you may use the OpenCV naming style, for example: DICT_6x6_250.
NOTE: The ArUco board consists on the **M** first markers of the chosen dictionary, where M = #rows x #columns.

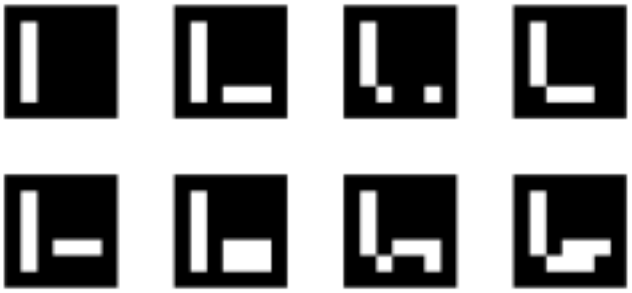

Figure 2.2: ArUco board generated using the first 8 markers of the original ArUco dictionary.

Part 3: Detection of ArUco Markers

During part 2you have used OpenCV to generate both a single ArUco marker and a board of them. You will now learn how to detect ArUco markers using C++ and OpenCV.

3.1 Detection of ArUco Markers

Implement a C++ program using OpenCV that allows capturing live video in an output window on screen. As you show either a single ArUco marker or a board of ArUco markers to the camera, the program should recognize all the markings in the video and, overlayed on each marker in the captured video, show the following:

- The marker contour highlighted with a green frame
- The upper left corner should have a little red square
- The identifier of each marker according to the chosen dictionary

The program must accept as an input parameter the dictionary that was used when generating the ArUco markers that are presented to the camera. This should make the detection process easier.

Figure 3.1 shows the aspect of an ArUco board with the information above overlayed.

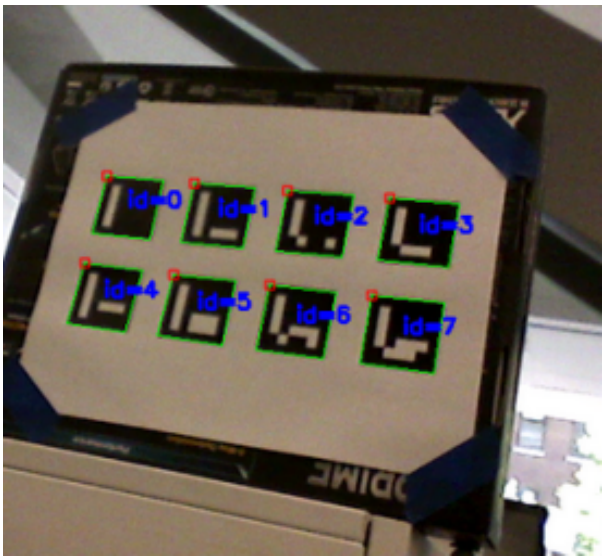

Figure 3.1: An example of how an ArUco board is detected

Part 4: Camera calibration using an ArUco board

During part 3 you created a program able to detect ArUco markers, which means obtaining their id within the used dictionary and estimating their positions within the image. In this next part you are going to use an ArUco board to calibrate cameras.

4.1 Camera calibration using ArUco boards and OpenCV

In a nutshell, camera calibration consists of the estimation of a mathematical relationship between the scene (3D world) and its projection onto the image (2D world). Several procedures exist for achieving such a goal, but here we will focus on the use of an ArUco board.

In order to achieve a successful camera calibration with a decent accuracy, a significant number of the ArUco board are required. Needless to say that the ArUco board images should be captured using the camera to be calibrated...

Typically between 20 and 30 images should be enough, as long as each of the images contains the very same ArUco board in different poses. You really need to capture images of the board from farther, closer, tilted, rotated, etc... different poses. Figure 1 shows 10 images out of a larger set of 30, captured with the goal of calibrating the camera that took the images. Use these as a guide of how you should capture the images of the ArUco board when trying to calibrate your camera.

Implement a program in C++ and OpenCV that allows capturing live video, showing the visible ArUco markers detected in real time. The program should be able to capture an image of the input video (no markings overlayed) every time the 'c' key is pressed. Every captured image will be added to the image set that will be used at the end of the capturing process (remember you need between 20 and 30 images in varied poses) to calibrate your camera. The image capturing process should finalize whenever the ESC key is pressed. As a rule of thumb, an image is considered as "good for calibration" when all the ArUco markers of the board have been correctly detected (i.e. when all ArUco markers appear with the overlays mentioned in lab assignment 3).

The program requires 7 parameters:

1. dictionary (alphanumerical)
2. detector parameters (alphanumerical)[1]
3. number of rows of the ArUco board (numerical)
4. number of columns of the ArUco board (numerical)
5. side length of a single ArUco marker in meters (numerical)
6. distance between ArUco markers in meters (numerical)
7. name of the calibration parameters to be saved in yaml format (alphanumerical)

The calibration parameters obtained will be saved in yaml format in a text file (with .yml or .yaml extension) that will be later used in lab assignment 5 (augmented reality). You are

[1] It's a text file in **yaml** format that OpenCV is able to parse. It' used to read and write parameters to and from disk.

strongly encouraged to get acquainted with what the calibration parameters in OpenCV are, and find their relationship with the existing state-of-the art.

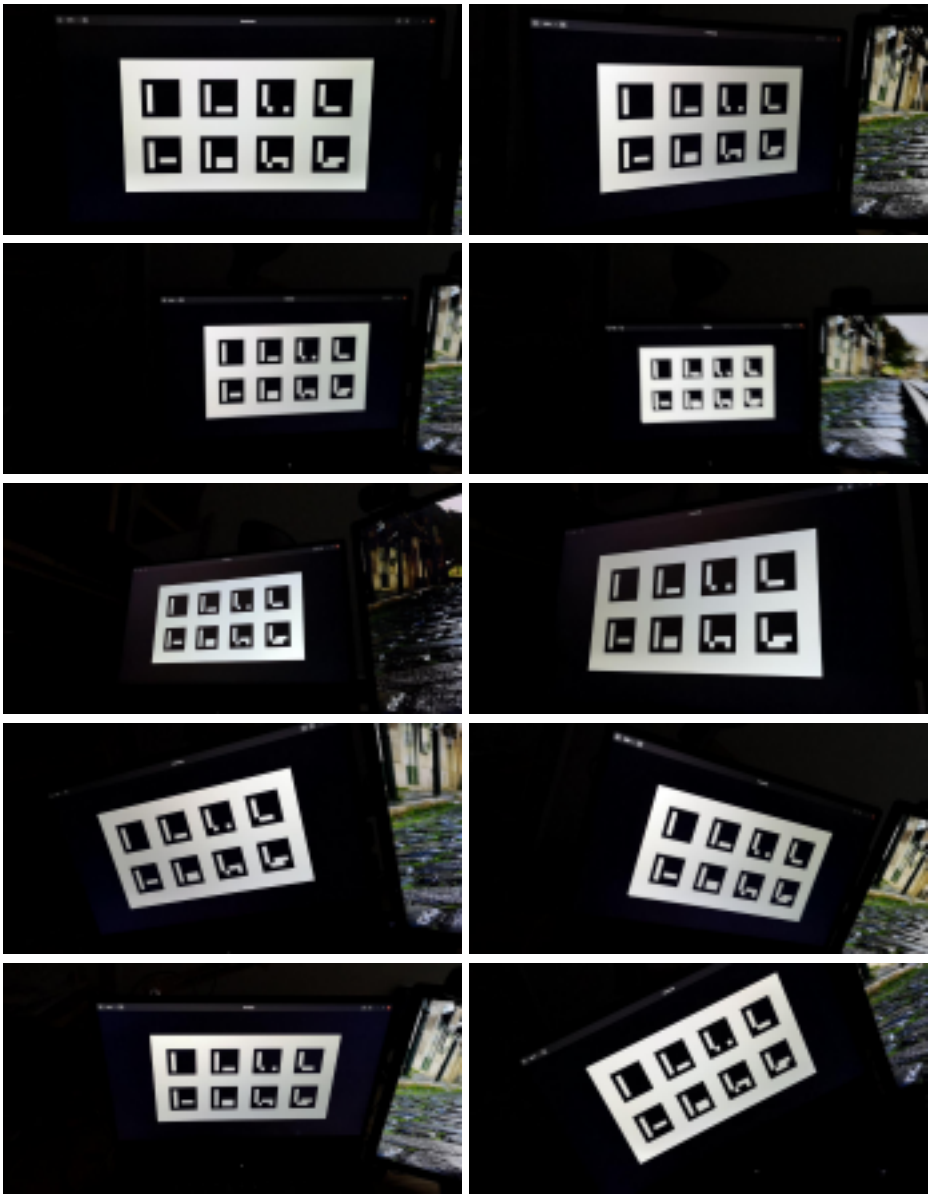

Figure 1: Examples of ArUco board images taken with the goal of calibrating the camera

Part 5: Using ArUco Markers for Augmented Reality and distance measurements

During part 4 you have created a program that calibrates a camera from a number of ArUco board images in varied poses. The calibration parameters were stored in a file with extension .yml or .yaml and they are going to be used here.

Now you will learn how to do augmented reality using ArUco markers and the camera calibration parameters obtained in the previous lab.

5.1 Augmented reality using ArUco markers

The seamless integration of real and virtual worlds in a visually credible manner is a technique widely used across numerous domains, including cinema, video games, robotics, architecture, and remote assistance. The breadth of applications has made augmented reality (AR) an active and important area of research.

Achieving convincing augmented reality fundamentally requires solving two core problems:

- Estimation of the pose of relevant objects in the scene
- Projection of virtual 3D objects onto the image

Using the mathematical model of a camera capturing real scenes, it is possible to estimate the pose of a known object within the environment. If an object can be reliably detected and its physical dimensions are known, virtual content can be projected onto the scene in a geometrically consistent manner.

The camera's mathematical model and its parameters is exactly what you obtain by calibrating the camera. And that is exactly the mathematical formula that obtains the coordinates of a 2D point in pixels taking as input the coordinates of a 3D point in metric units.

To illustrate, consider a simple example: a table is present in a scene, and the dimensions of its surface (width and depth) are known. If a program can detect the table's surface and estimate its pose, a virtual 3D object can be projected into the image so that it appears to rest naturally on the table.

There are two ways for doing augmented reality:

1. Using fiducial markers attached on the objects to be recognized
2. Without fiducial markers, using artificial intelligence in order to recognize the objects in the scene.

In this lab you're going to focus on the first option, as you can imagine, using well known ArUco markers for detecting their own pose and, furthermore, to "draw" a simple 3D object on top of them.

Implement two C++ and OpenCV command line programs according to the requirements below:

5.1.1 Program implementation: pose_estimation

- The program detects the pose of an ArUco marker present in a live video, captured with your webcam.

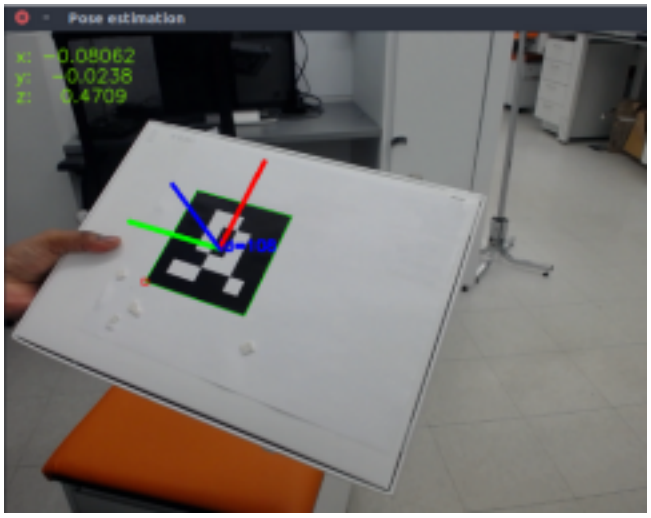

Figure 1: Example of an ArUco marker's pose detection

- On the ArUco marker, draw a green frame surrounding it. As well, draw a small red square on one of its corners. The marker id within the dictionary it belongs to should also be overlayed on the ArUco marker.
- In the middle of the marker draw a coordinate system consisting of 3 segments: a blue one (normal to the maker's plane -Z-), a red one (parallel to one of the sides of the marker -X-) and a green one -Y- at 90º from X, contained in the marker's plane.
- The values of the X, Y and Z coordinates of the coordinate system's center should be overlayed on the screen.
- All overlayed items should be "live" on the input video, following the pose of the ArUco marker and updating X, Y and Z values in real time (see Figure 1).
- The program requires 3 parameters:
    1. dictionary (alphanumerical)
    2. id of the marker within the chosen dictionary (numerical)
    3. length of the marker's side in meters (numerical)

5.1.2 Program implementation: draw_cube

This program accepts the very same parameters than pose_estimation:

1. dictionary (alphanumerical)
2. id of the marker within the chosen dictionary (numerical)
3. length of the marker's side in meters (numerical)

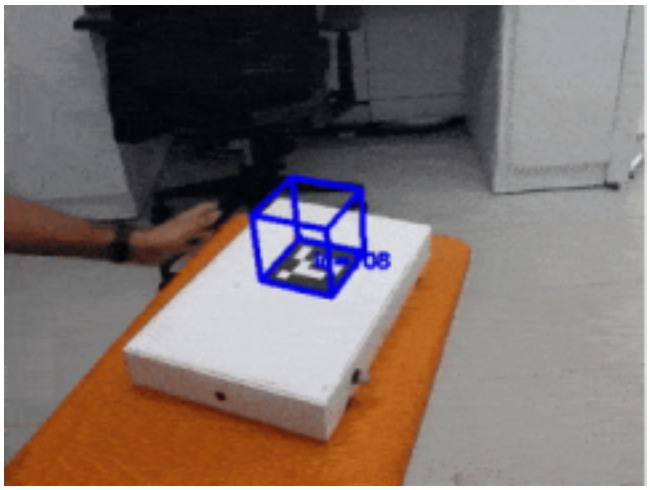

Figure 5.2: Example of a synthetic cube overlayed on top of an ArUco marker

But this time you don't need to overlay the coordinate system nor the green frame nor the red square. What you are required to draw this time is a cube of length equal to the ArUco marker's side length.

The cube must be overlayed on the live video coming from your camera whenever an identified ArUco marker is within the camera view and it should follow the marker's pose in real time. Figure 5.2 shows an example of how it all must look like.

5.2 Distance Estimation using ArUco markers

ArUco markers are extremely useful to compute real distances between detected fiducial markers. In this last part, you will be working with two ArUco markers placed in two different locations (not part of the same grid/printed A4 paper). However, they should be close enough so that you can capture an image where both markers are visible simultaneously.

5.2.1 Program implementation: relative_pose_estimation

This program will be an adaptation of program 5.2.1

- The program detects the poses of two ArUco markers present in a live video, captured with your webcam.
- On each of the ArUco markers, draw the green frame surrounding it. Additionally, draw a small red square on one of its corners. The marker id within the dictionary it belongs to

should also be overlaid on the ArUco markers.

- In the middle of the markers draw a coordinate system consisting of 3 segments: a blue one (normal to the maker's plane -Z-), a red one (parallel to one of the sides of the marker -X-) and a green one -Y- at 90º from X, contained in the marker's plane.
- Compute the relative position of the right-most marker with respect to the left-most marker by computing a coordinate transformation on the pose estimates of the ArUco markers.
- The values of the X, Y and Z relative coordinates should be overlayed on the screen.
- All overlayed items should be "live" on the input video, following the pose of the ArUco markers and updating X, Y and Z values in real time (similar to Figure 1).
- The program requires 3 parameters:
    1. dictionary (alphanumerical)
    2. ids of the markers within the chosen dictionary (numerical)
    3. length of the marker's side in meters (numerical)

5.2.2 Program implementation: distance_estimation

This program will be an extension of the previous one.

This program accepts the very same parameters than pose_estimation:

4. dictionary (alphanumerical)
5. ids of the markers within the chosen dictionary (numerical)
6. length of the marker's side in meters (numerical)

But this time you don't need to overlay the coordinate system nor the green frame nor the red square. What you are required to draw this time is a line between the ArUco's centres and display the euclidean distance between the ArUco markers instead of the X, Y and Z relative coordinates.

The line must be overlayed on the live video coming from your camera whenever both identified ArUco markers are within the camera view and they should follow the markers' poses in real time (similar to Figure 5.2).

Once you have completed these two programs, validate the correctness of your system by using a ruler to measure the relative pose and euclidean distance between the two ArUco markers in the real world.